

Computational complexity

Algorithms

Spring 2026

How difficult is a problem?

- Computational Complexity
- For simplicity, two classes: Easy vs Hard

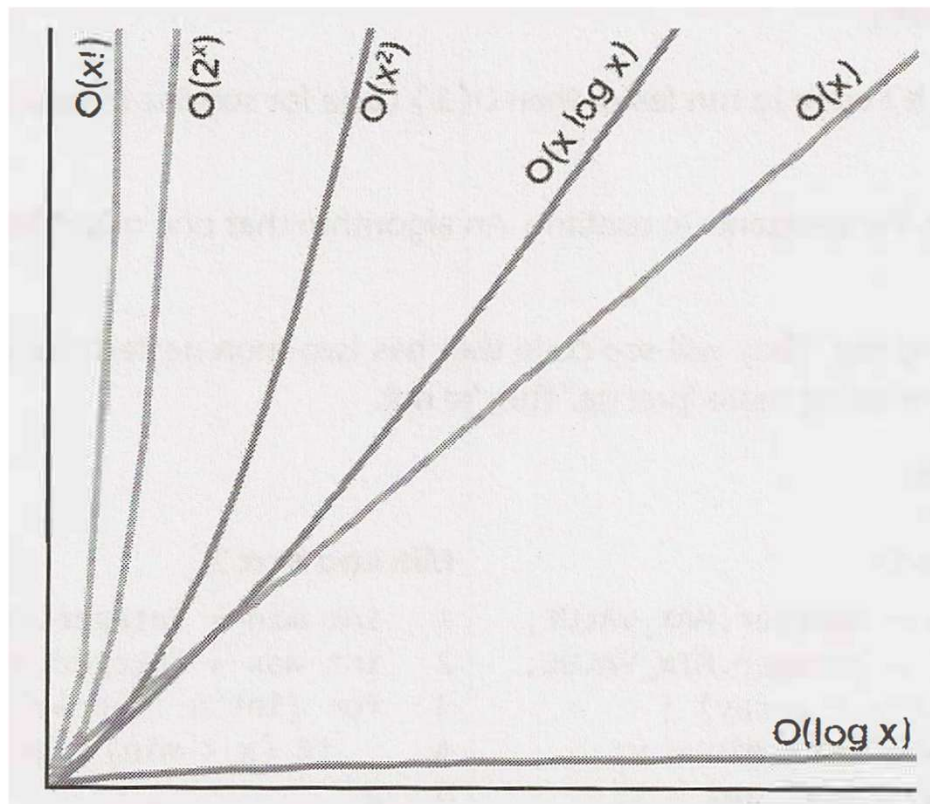
Tractable vs Intractable problems

- How to know if a problem is hard or intractable?

Easy and Hard problems

- Easy → Can be solved with a *polynomial time* algorithm
sorting, searching, finding shortest paths.
polynomial time means $O(n^d)$ where n is input size and d is a constant
like $O(n^2)$, $O(n^3)$, $O(n^4)$, etc.
- Hard → Requires exponential time in the worst case.
 $O(2^n)$ or $O(n!)$

Identify Exponential growth rate



Pop Quiz

- There are n words in a document. n is input size which can vary depending on the size of the document.

My algorithm takes n^{1000} comparisons among the words to solve the problem.

My algorithm is

- a) Polynomial time algorithm
- b) Exponential time algorithm

Broadly, two complexity classes

- P = Problems solvable in Polynomial time
- NP = Decision problems solvable in non-deterministic polynomial time

NP-hard Problems

- I can't find an efficient algorithm, because no such algorithm is possible!
- Example: Travelling salesman problem

Importance in internet security

- Encryption algorithms allow computers to securely communicate
- Prime factors and internet security

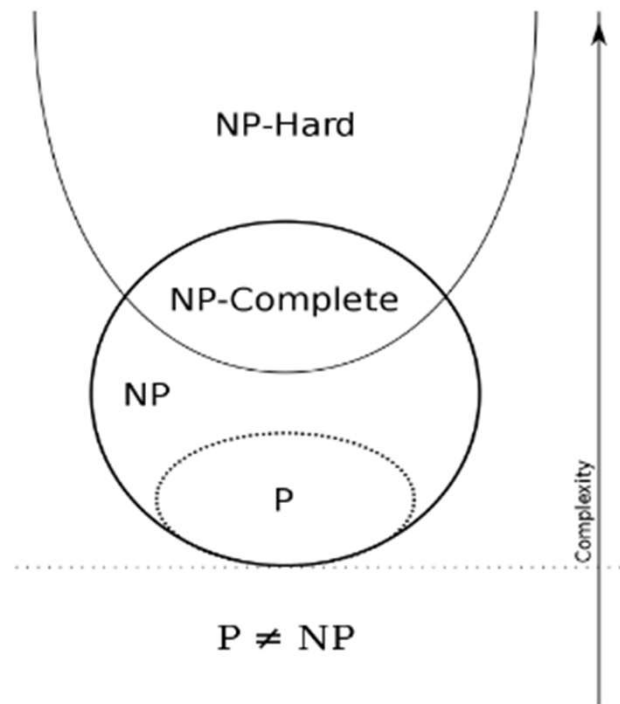
Factoring large integers into prime numbers

$$70 = 2 * 5 * 7$$

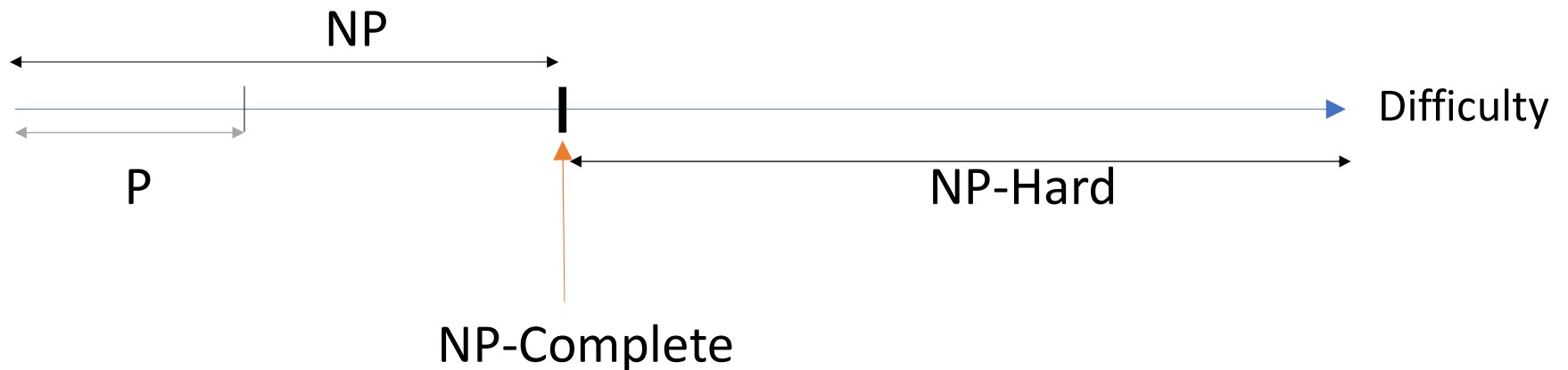
Security on the internet depends on prime factorization being not easily solvable.

Computational Complexity Classes

- NP stands for
Non-deterministic
polynomial time

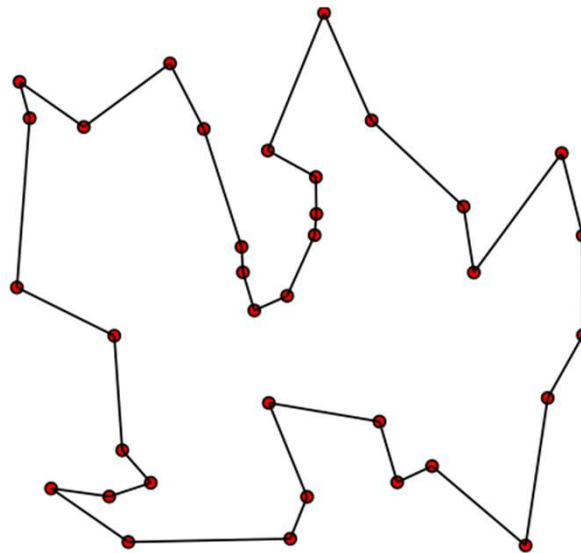


P, NP, NP Complete and NP-Hard



Example of hard problem: Traveling salesman problem

- Given a list of cities and the distances between each pair of cities, what is the shortest possible route that visits each city exactly once and returns to the origin city?

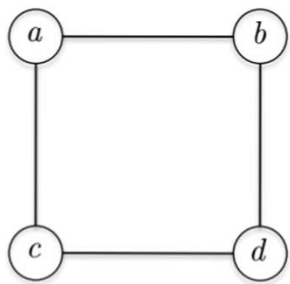


Source: Wiki

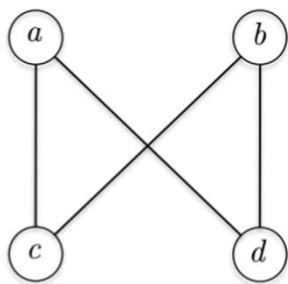
Travelling salesman problem

- Find tours among cities a, b, c, d
- Use Brute Force or Exhaustive search of all options

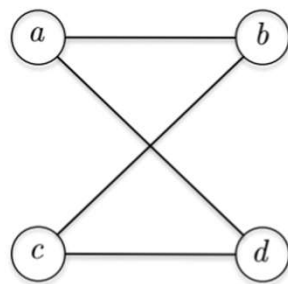
a b d c a



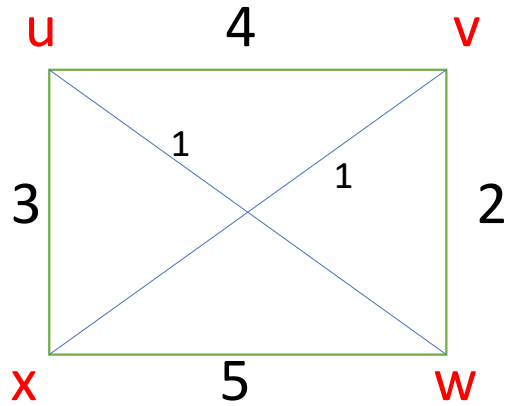
a d b c a



a b c d a



Minimum length Tour for the Salesman?



Answer

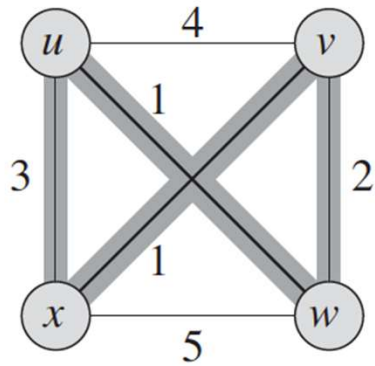
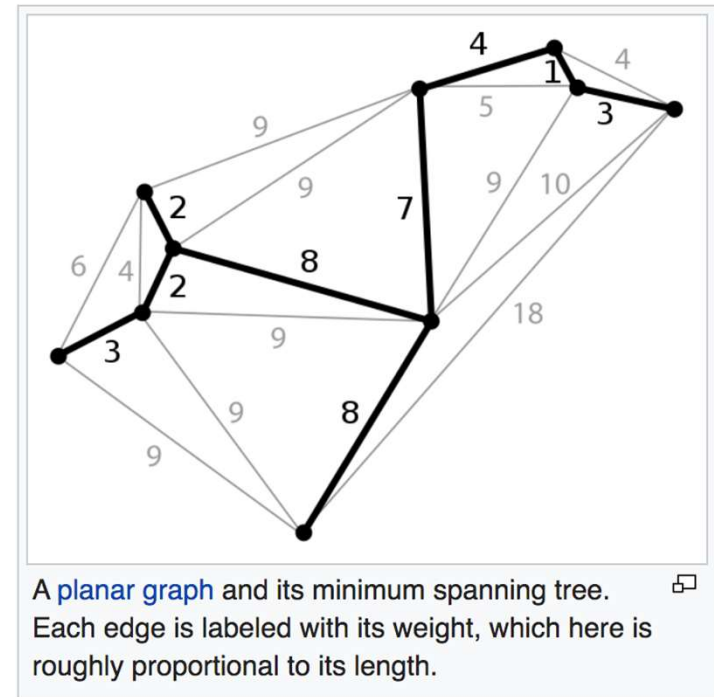


Figure 34.18 An instance of the traveling-salesman problem. Shaded edges represent a minimum-cost tour, with cost 7.

Comparing TSP with Min Spanning Tree

- Make the locally optimal choice at each stage
- Min Spanning Tree algorithm is greedy



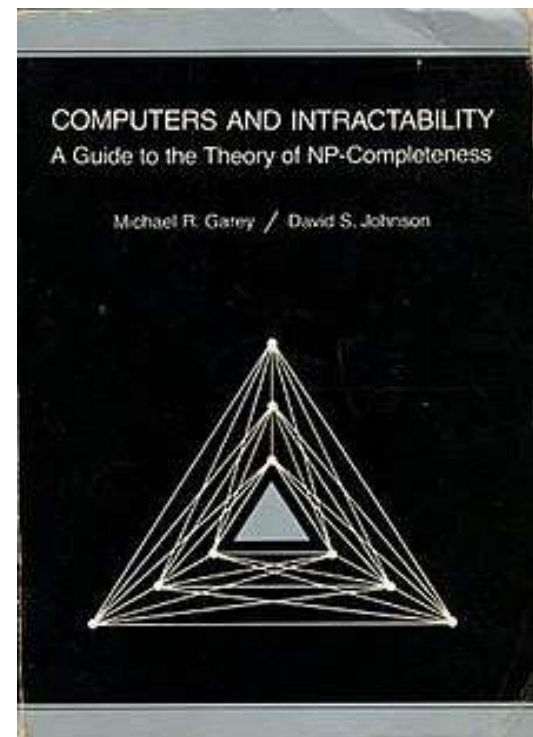
Two classes of NP

- NP-hard:
optimization problems, return minimum/maximum among all.
e.g., Find the minimum length tour for travelling salesman problem
- NP-Complete: decision version of NP-hard problem
Yes or No answer.
e.g., Is a tour of length at most 100 possible or not?

Computers and Intractability

- Garey and Johnson book

Appendix with over 300 NP-hard computational problems.



Pop Quiz

Select True or False

- Problems in class NP can not be solved (T/F)
- Problems in class NP can be solved by exhaustive search (T/F)

Multiple choice question

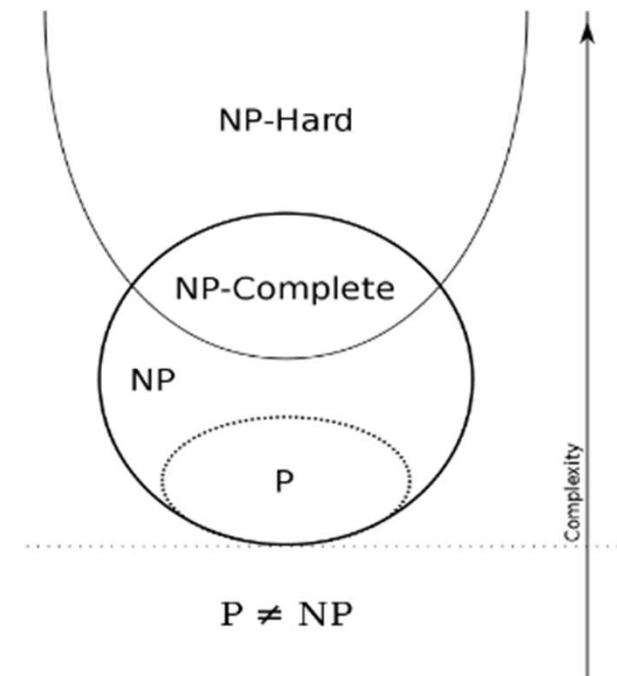
Is there a path between point A and point B in at most 14 hops?

- A) This is a decision problem
- B) This is an optimization problem

Complexity class NP

Second definition

Decision problems for which any solution can be verified as a solution in polynomial time

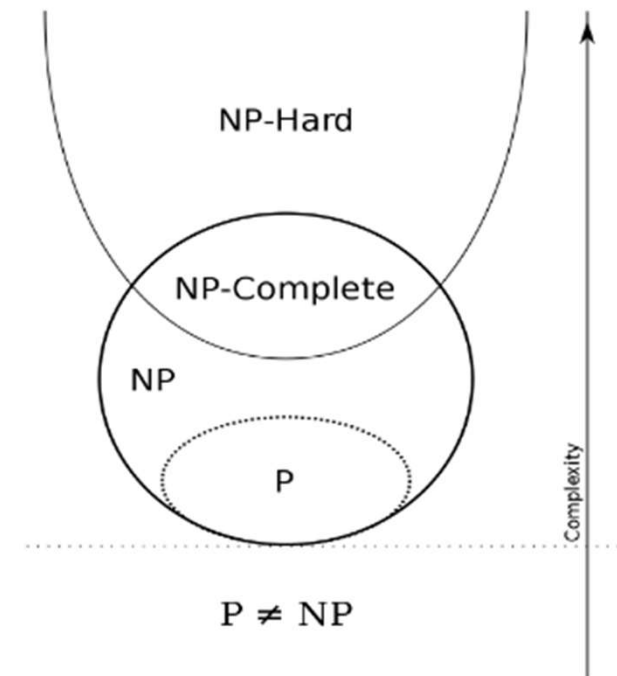


Complexity class NP

Second definition

Decision problems for which any solution can be verified as a solution in polynomial time

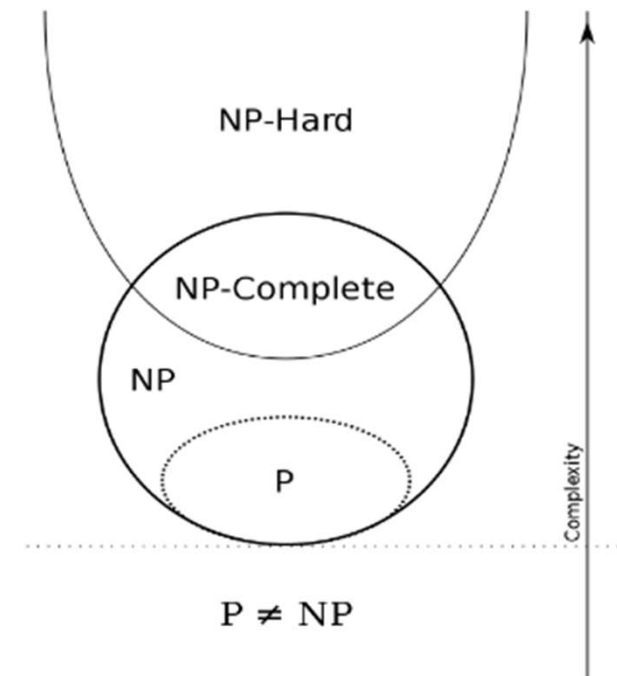
Example: Integer prime factorization
easy to check if ($a * b == c$)



Condition to be in NP

Polynomial time verification

If solution is provided to you, then
you can verify
that the solution is correct
in polynomial time



Is $P \neq NP$?

or

Is $P = NP$?

Unsolved problem in Theoretical Computer Science

Million dollar prize [\[Link\]](#)

[Link](#)

$$P \neq NP$$

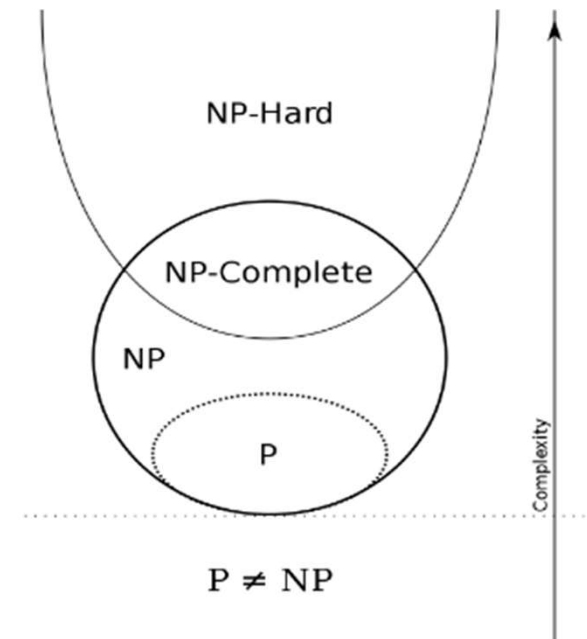
Checking a solution to a problem
can be fundamentally easier
than coming up with a solution from scratch.

Creating a proof vs checking a proof

Writing a program vs checking it

NP-complete

- Class of decision problems which contains the hardest problems in NP.
- Each NP-complete problem is in NP.



Travelling salesman (TSP) problem

- Find the shortest tour for a travelling salesman.

This Optimization Version is **NP-Hard**

It is NOT in NP: To verify that a specific tour is the *shortest* possible tour, you would (theoretically) need to check it against all other possible tours.

There is no known way to verify "optimality" in polynomial time.

NP-Hard Problems

- A computational problem is NP-hard if every problem in NP reduces to it.
- Reduction

NP-Hard Problems

- A computational problem is NP-hard if every problem in NP reduces to it.

How to Prove a Problem Is NP-Hard

To prove that a problem B is NP-hard:

1. Choose an NP-hard problem A .
2. Prove that A reduces to B .